Repro Radar - Final Report

Status: written (#120). *Each section lists its sources. Every number comes from a script (generated blocks, checked by* `python scripts/report_numbers.py --check`*), and every factual statement is listed in a claim to source table (*`docs/INTRODUCTION_SOURCES.md`*,* `docs/report/RELATED_WORK_SOURCES.md`*). The report was read through and made consistent on 2026-10-20 (sections 1, 2, 3, 4, 6.4a, 6.5a to 6.7, 7, 9, and 10): the numbers against their stored files (*`python scripts/report_numbers.py --check`*), and the related-work references against Crossref and arXiv (*`scripts/verify_sources.py`*, 9 of 9 verified). It was read through again on 2026-10-25 (#503): the results of the last days (sections 6.4, 6.11), the two claims of the report (section 1), and the limitations were brought up to date, and the contradictions and stale sentences that were found are listed in* `docs/PAPER_STATUS.md` *(section 2). Which parts are final and which wait for the owner is also there (section 1). A last read-through on 2026-10-28 (#550) placed the label rule (#540), the Tier 3 binding (#542), the failure statistics (section 6.12) and the final dashboard (section 6.13), and extended the limitations (section 7); its findings are in* `docs/PAPER_STATUS.md` *(section 2).*

---

1. Abstract

Repro Radar is a pilot tool that tries to reproduce published results mechanically: it runs a paper's released code at a pinned commit in an isolated Docker sandbox, extracts the paper's own numerical claims with a quote check, and scores it on a four-tier rubric. We applied it to six tabular papers in three pilots, one counted run per paper, and then to six further papers in a fourth pilot.

Install success: 6 / 6 (1.0) (Pilot 1: 1 / 6 (0.167); Pilot 2: 6 / 6 (1.0)). Run success: 2 / 6 (0.333) (Pilot 1: 0 / 6 (0.0); Pilot 2: 1 / 6 (0.167)). Pilot success: 2 / 6 (0.333) (Pilot 1: 0 / 6 (0.0); Pilot 2: 1 / 6 (0.167)).

The third pilot's settings were chosen after the second pilot's failures were seen, so the change in the rates measures those settings, not the papers. The stops that remain are the data, the network, and memory. The fourth pilot ran six papers that the tool had not seen from their README commands alone, with a lower install rate than Pilots 2 and 3, whose run setups the project had prepared by hand (section 6.5). No run was shown to reproduce an accepted claim: before the claim map (#507, #509) the agreement tier never matched a real claim through the command line; with the map the one stored run that prints a number for an accepted claim (`paper_003`: one claim, one number) is compared and is close to it, 3.05 % apart, which is not a confirmed reproduction (section 6.11). With six papers and one run each, the results are anecdotal. The contribution is the method, a sandbox checked against real Docker, and a measured list of what stops each paper.

The report makes two claims and keeps them apart. The main claim is that the tool gives the right decision on a known example: on a synthetic paper and on hand-made claims moved from real runs' own values the decision follows the numbers as designed (sections 6.4 and 6.9); on real papers the true answer is not known, and the one real run whose model, dataset and metric match an accepted claim is close to it but is not a confirmed reproduction (section 6.11). The second claim is a pilot estimate of how many papers run, with wide Wilson intervals because the samples are small and selected (sections 6.5, 6.6 and 6.11). Two searches for a real run whose output matches its paper's table, a list of candidates and an automatic search, found none (sections 6.8 and 6.10); this is a finding about README commands, which mostly do not produce the paper's table, and not a success rate.

Since the third pilot the project added five things, each reported with numbers generated from stored files: the stability tier now runs a paper once per seed (section 6.4a), a synthetic paper with a modern PyTorch was run with the local GPU through the same pipeline (section 6.4a), an exploratory analysis covers all twelve papers of Pilots 1 to 4 (section 6.6), a rule-based diagnosis layer explains a stored run and the difference between two runs, and its classifier was checked against hand-written labels (section 6.7), and an attempt to find independent reference articles in ReScience C selected none (section 6.5a). The diagnosis layer was not tried with a real language model.

Four later changes are reported in sections 6.11 to 6.13. A run in which no accepted claim could be compared with a number of the run is labeled `Needs Review` and never `Verified` or `Partially Verified` (#540), because its Tier 2 is a neutral baseline and not evidence. The stability tier follows the metric that an accepted claim names, when the claim map names one (#542; a default chosen by the assistant, which the owner may revise). The first failure of every stored run was counted by category, with a selection-bias warning (section 6.12). And the local dashboard now states, beside every label, whether a claim was compared and why the label

was given (section 6.13).

- **Sources:** sections 5 to 7 of this report; the generated sentence above is copied from `data/pilot3_runs/comparison.json` by `scripts/report_numbers.py`.
- **Status:** written.

2. Introduction

Reproducibility in empirical machine learning rests on being able to re-run a published result from the paper and its code. The point is not new: Claerbout argued in the early 1990s that a computational result's real scholarship is the full software environment that produced it, not the paper describing it [1]; Buckheit and Donoho later put this plainly, that an article is advertising for the scholarship, the scholarship itself is the code and data [2, 3]. Tool and practice collections followed [4], and so did evidence that the problem had not gone away: a 2016 *Nature* survey described a "crisis" in research reproducibility across fields [5], and a 2019 NeurIPS program examined the state of reproducibility in machine learning specifically [6]. In deep reinforcement learning, reproducing a published result is "seldom straightforward," because results vary with non-deterministic benchmark environments and with variance intrinsic to the methods themselves [7] - the same caution applies more broadly: a result can differ between runs for reasons that have nothing to do with whether the original claim was sound.

This project's own seed corpus illustrates the problem at a small scale. Of the first 16 numerical claims recorded from 6 papers, only 2 could be found, as printed, in the paper they were attributed to; the rest were either not present or attached to the wrong paper or table (`docs/CORPUS_VERIFICATION.md`). Two of the six repositories had been renamed or had a target script removed by a later rewrite of the project they belong to. Re-running a paper is therefore not only a matter of running code: it requires reconstructing the right environment, the right version of the code, and the right data, and checking numbers against the paper itself rather than against a value someone wrote down from memory.

Repro Radar is an attempt to do this mechanically and honestly for a small set of tabular machine learning papers: stage each paper's code at a pinned commit, run it inside an isolated sandbox whose isolation properties were checked against a real Docker daemon in continuous integration while a CI workflow existed (it was removed in #388, `docs/DECISIONS.md` D-010; `docs/SANDBOX_ACCEPTANCE.md`), extract the paper's own numerical claims with a verbatim-quote guard against invented matches, and score the result on a four-tier rubric instead of a single pass/fail. As of this writing, the sandbox and the corpus verification are checked, the six papers of the corpus have been run through the sandbox in three pilots (section 6), and `cli.py score` scores a stored real run against accepted claims (`docs/gap_analysis.md`, finding GA-2, closed by

136); but no run has been shown to reproduce an accepted claim; the scoring rubric's agreement tier has been exercised on real results

for one claim only, `paper_003`, through the claim map (#507, #509; sections 6.3 and 6.11). The stability tier has been exercised on real seed runs of two of the papers (section 6.4a). The seed pilots cover tabular ML benchmarks only; the later pilot, the candidate lists and the automatic search drift to other kinds of papers (sections 6.5, 6.8 and 6.10), but no claim is made about other scientific fields.

- **Sources:** [1]-[7] and every other factual statement above are listed with how they were verified in `docs/INTRODUCTION_SOURCES.md`, sections 1-2. That file also records what an earlier draft claimed and could not source, and was corrected to remove (section 3) - for example an unsourced reproducibility percentage, a misdated quotation, and a claim that the project covers "all scientific fields," which it does not.
- **Status:** written, from existing sourced evidence.

3. Related work

Work on reproducibility in machine learning can be grouped by what it measures: how well papers document their experiments, whether independent re-implementation works, whether released code runs, and how much

results vary from run to run.

Documentation and re-implementation. Gundersen and Kjensmo surveyed 400 papers from IJCAI and AAAI with six reproducibility metrics; none of the papers documented all the variables needed to reproduce its results, and between 20% and 30% of the variables for each factor were documented [8]. Raff attempted to implement 255 papers published between 1984 and 2017, without looking at the authors' code, and analyzed the recorded features of each paper statistically, on the argument that releasing code is important but not sufficient for determining reproducibility [9]. At the level of a community, the NeurIPS 2019 reproducibility program combined a code submission policy, a reproducibility challenge, and a reproducibility checklist in the submission process [6]. Collberg and Proebsting argue, for computer systems research, for funding repeatability engineering and rewarding the sharing of research artifacts [10].

Executing released code. The studies closest to Repro Radar's question run code that authors have released. Trisovic et al. executed R code from more than 2,000 replication datasets in a clean runtime environment: 74% of the R files failed to complete without error in the first execution, and 56% still failed after automatic code cleaning [12]. Pimentel et al. report a large-scale study of the quality and reproducibility of Jupyter notebooks [11]. Boettiger examines why code written for one research project often cannot be run or extended by later researchers, and presents Docker as one way to address this [13]. Repro Radar also uses containers, but as an isolation boundary for code it does not trust as well as for repeatability (section 4.1).

Variance. Henderson et al. show, for deep reinforcement learning, that non-determinism in benchmark environments together with variance intrinsic to the methods can make reported results hard to interpret [7]. Bouthillier et al. model the benchmarking process and find that data sampling, parameter initialization, and hyperparameter choice each affect results markedly [14]. These results motivate Tier 3 of the score, stability across seeds (section 4.2); the counted pilot runs were single runs, and the seed runs of section 6.4a are what exercise that tier.

Where Repro Radar sits. Compared with the large studies above, Repro Radar is small and deep: six tabular machine learning papers chosen by the project, not sampled, each at a pinned commit, run in an isolated sandbox with a setup written by the project, and judged against numbers that a person accepted after a verbatim-quote check. It does not estimate how reproducible a field is, and its counts are not comparable with those of [12]: other language, other selection, far smaller n. What it contributes is a method and a measured list of what stops a paper from installing and running (section 6).

- **Sources:** [6]-[14]. [6] and [7] were verified earlier (`docs/INTRODUCTION_SOURCES.md`); every source and every statement in this section is listed with its evidence in `docs/report/RELATED_WORK_SOURCES.md`. Each source was looked up by `scripts/verify_sources.py` in Crossref or the arXiv API (`docs/report/related_work_verification.json`); **only bibliographic data and abstracts were read, not full texts**, and for [11] only the title and bibliographic data (no abstract was returned).
- **Status:** written.

4. Method

4.1 Sandbox and threat model

Third-party paper code is untrusted by construction: it is run once, usually without being read first. Every run happens inside a Docker container built from `docker/Dockerfile.sandbox`, under a non-root user (uid 1000, verified check 1 of `docs/SANDBOX_ACCEPTANCE.md`), with the network disabled (`--network none`, check 2), the root filesystem read-only (check 3), and `/tmp` (64 MiB) and the user's home directory (2048 MiB) as size-limited `tmpfs` mounts, both `noexec` (`docs/DECISIONS.md` records why the home directory stays `noexec`: compiled dependencies are loaded from a separate read-only directory, `/workspace/deps`, check 7). Resource limits are enforced and checked against real load: 2 CPUs (check 17), 6-8 GiB of memory with the container killed and the kill reported as `OOM_KILLED` rather than confused with a timeout (checks 8 and a dedicated acceptance test), 100 processes (check 9), and a wall-clock timeout per phase (check 4). Dependencies are installed from pre-downloaded binary wheels only - building a source distribution would run arbitrary `setup.py` code outside the sandbox, so source-only packages are either built inside a separate, identically isolated build-phase container (check 18) or flagged `NEEDS_SDIST_BUILD` and not built at all (`docs/gap_analysis.md`, GA-22). A GPU is never attached unless explicitly requested (check 20) and is never part of the counted pilot run (`docs/PILOT_2.md` section 3); one run of a synthetic paper with the GPU attached is reported in section 6.4a. Running without Docker at all is refused by default; an explicit, logged opt-in is required to fall back to an unisolated local

run (check 6). The container shares the host kernel, so these measures reduce, not eliminate, the risk of running unknown code - this is a property of container isolation generally, not specific to this project.

4.2 Reproducibility score

Each run is scored on four weighted tiers (`src/config.py`): **Tier 1**, build and install success, 35%; **Tier 2**, agreement between the paper's claimed metrics and the reproduced ones, 35%; **Tier 3**, stability across repeated runs with different seeds, 15%; **Tier 4**, documentation and license completeness, 15%. The composite score classifies a paper as `Verified` (total >= 85, and also Tier 1 >= 90 and Tier 2 >= 80), `Partially Verified` (total >= 60 and not `Verified`), or lower tiers below that (`src/scoring.py`; a run with Tier 2 < 50 and Tier 1 >= 80 inside that band is `Needs Review`). A failed build or run is `Failed`. A run in which no accepted claim was compared with a metric of the run (none accepted, none with a counterpart, or none found in the output) is labeled `Needs Review` whatever its total, because Tier 2 is then the neutral baseline of 70 and not evidence (#540); the total still contains that baseline.

Tier 2 compares each claimed metric with the reproduced one using a single default tolerance of 5% (`DEFAULT_TOLERANCE`, `src/config.py`), applied in `src/scoring.py`: a reproduced value within 5% relative error earns full points; for a metric claimed inside [0, 1] (accuracy, AUROC, F1) an absolute difference of up to 0.05 also counts; within twice the tolerance earns half the points; beyond that, none. Runtime and latency metrics, recognized by name, are judged by relative error only, because an absolute bound would let 0.02 s become 0.06 s, a 200% error, and still pass. The scorer has no per-category tolerance (for example a tighter bound for AUROC than for accuracy) and does not separate a hardware difference from a real divergence; a module that was meant to do this was never connected to the scorer and was removed (#254). The comparison of two runs in the diagnosis layer (section 6.7) lists recorded differences and sizes a metric shift, but the scorer does not use it. A claim is matched with the run's metrics by exact name; an accepted claim is keyed by its question ID, which is not the name the metric extractor gives the metric, so until the claim map (#507, #509) a real claim was reported as not found; `cli.py score` now pairs the two through `data/claim_metric_map.json`, and a claim for which the run has no counterpart stays out of Tier 2 (section 6.11). Tier 3's stability check uses the coefficient of variation (CV) across seeds: CV <= 2% is scored as deterministic, <= 5% as stable, with the score falling as the variance grows. Since #48 a paper that has a seed parameter of its own is run once per seed (`docs/MULTI_SEED.md`, `scripts/run_multi_seed.py`). Since #542 the scorer reads the stored seed record (`cli.py score`, the dashboard database, `scripts/real_verdicts.py`): if an accepted claim is mapped to a run metric that has seed values, that metric decides Tier 3 (a default chosen by the assistant, which the owner may revise, `docs/MULTI_SEED.md` section 5a); otherwise the metric with the largest CV among the metrics found in every successful seed run decides. A paper without two successful seed runs keeps the single-run baseline of 75.0, which is a placeholder and not a measurement.

4.3 Claim extraction

A claim is one number a paper reports for one metric, one dataset, one model, and one split, defined precisely in `docs/CLAIM_SCHEMA.md` (W1, #129): the value is kept exactly as printed (`value_as_printed`, including `%`, `±`, and any stray spacing inside the number) alongside a normalized value computed by `src/claim_schema.py`, never typed by hand. Extraction reads the PDF's page text, cached by the file's hash (W2, #130, `docs/PDF_TEXT_EXTRACTION.md`). No API key is currently used (decision D-004, `docs/DECISIONS.md`); instead, an agent reads the paper text under a fixed set of rules and answers one prepared question per claim (`docs/AGENT_EXTRACTION.md`): answer only from the given text, never guess or borrow a value from a similar dataset or metric, and answer `not_found` with a reason when the paper does not report it. Two extraction passes are compared by normalized value (D-005); every claim's quote must also occur verbatim in the page text, checked by the quote guard (W4, #132, `src/quote_guard.py`) - a claim whose quote cannot be found is rejected, not silently dropped. Accepted claims are stored with their page, quote, and extraction method (W6, #10), and nothing reaches scoring until a person has reviewed it one by one (W7, #134, `src/claim_review.py`, `python cli.py review-claims`). Measured against the project owner's own gold set of 18 questions over 6 papers, this pipeline reached precision 1.0 and recall 1.0 (W5, #133, `data/claims/gold_score_pass_A.json`), including two cases built specifically to catch a fabricated match: a dataset name ("EP3602") and a metric ("HIGGS") that do not occur in their papers at all, both correctly answered `not_found`.

- **Sources:** `docs/SANDBOX_ACCEPTANCE.md` (all numbered checks cited above are its own verbatim evidence, taken from a CI run against a real Docker daemon, 2026-10-05); `docs/DECISIONS.md` (D-004, D-005, and the home-directory `noexec` decision); `src/config.py`, `src/scoring.py` (weights, thresholds, tolerance rules, CV

bands - read directly from the code, not retyped by hand); `docs/CLAIM_SCHEMA.md`, `docs/PDF_TEXT_EXTRACTION.md`, `docs/AGENT_EXTRACTION.md`; `data/claims/gold_score_pass_A.json` (the precision/recall numbers and the two regression cases).

- **Status:** written. This section describes the design and is not a results claim: `cli.py score` can score a stored real run (#136), but no run has been shown to reproduce an accepted claim yet (sections 6.3 and 6.11).

5. Experimental setup

5.1 Corpus

The seed corpus went through two corrections before reaching its current 6 papers. The first 18 entries, written in Week 1, were placeholders that in several cases paired a paper's title with an unrelated general-purpose library's repository (`docs/PROGRESS.md`). These were replaced by 8 entries on 2026-09-13, but that replacement was itself called "verified" without every entry having actually been checked; a later review found only one paper had been (`docs/PROGRESS.md`, the 18->8 note). Two further entries were then dropped as not fixable, leaving the 6 papers scored in `docs/CORPUS_VERIFICATION.md` today: TabPFN, FT-Transformer, TabR, FastSK, SAINT, and DANet, each pinned to one commit of its repository.

An automated check (`scripts/verify_corpus.py`, run in CI on every push while the workflow existed, now run locally, D-010) verifies, for every entry, that the paper title matches its arXiv or Crossref record, that the repository exists, that its declared license matches GitHub's own license detection and is on an allow-list of open licenses, and that the pinned target script exists at the pinned commit. As of the last recorded run, 6 of 6 entries pass this check. Passing it proves the links, license, and script file are real; it does not prove the script runs or reproduces the paper's numbers - each entry's row in `docs/CORPUS_VERIFICATION.md` separately tracks GPU requirements, whether its data must be downloaded, and how many of its recorded claims could actually be found in the paper (as low as 0 of 3 for one entry). No entry in the corpus is called "verified" as a whole; every one still carries at least one open, explicitly marked item.

5.2 Sandbox limits and run setup

Every run of Pilots 1 and 2 uses the same fixed resource limits: 2 CPUs, 6 GiB of memory (6 GiB swap), 100 processes, and a 3600-second timeout per phase (build, install, run), with the network disabled throughout (`docs/PILOT_2.md` section 3); Pilot 3 changed four per-paper settings (section 6.1a) and Pilot 4 used 50% of the host's memory (section 6.5). The first pilot (Pilot 1, `docs/PILOT_WEEK2.md`) ran each paper's script directly from the repository root with no paper-specific adjustment; it showed that no paper reached its own experiment code this way (full accounting in section 6). Each paper's `run_setup` in `data/seed_papers.json` now records, individually, the fix that pilot found necessary: the entry command (several papers are driven through a Jupyter notebook via `nbconvert`, not a plain script), the working directory, whether the repository needs a writable per-run copy, which datasets must be fetched to a local cache before the run (since the sandbox has no network), and which packages must be built from source inside an isolated build phase rather than installed as a published wheel. This is also where each paper's Python version comes from (3.8, 3.9, or 3.11, chosen from its own README or environment file) and where any extra install-time requirements or version constraints are recorded, each with the reason and source for the choice next to it - not applied silently.

5.3 Success definitions

Three outcomes are defined precisely and used consistently across both pilots (`docs/PILOT_WEEK2.md` section "Definitions"):

Term	Definition
Install success	dependency_install_status: INSTALLED in the run report. FAILED, SKIPPED (nothing to install), and UNKNOWN do not count.
Run success	Exit code 0 of the run phase, in a run that was actually valid (staged at the pinned commit, isolated, Docker used). TIMEOUT, OOM_KILLED, and any other non-zero exit code do not count.
Pilot success	Install success and run success, together.

Every rate (install success rate, run success rate, pilot success rate) is reported with a denominator of 6 - every paper in the corpus, including one whose staging or run cannot even start - so a paper is never quietly excluded from the count. A paper without pilot success is assigned exactly one failure category, decided by the

first failure in the order install -> run, with the log line that decided it quoted in the pilot report.

- **Sources:** `docs/CORPUS_VERIFICATION.md` (per-entry status and checks); `scripts/verify_corpus.py` and its CI job (automated checks); `docs/PROGRESS.md` (the 18->8 and 8->6 corpus history); `data/seed_papers.json` (`run_setup`, `python_version` fields); `docs/PILOT_WEEK2.md` and `docs/PILOT_2.md` (limits and definitions).
- **Status:** written.

6. Results

6.1 Can the papers be installed and run? Pilot 1 and Pilot 2

Both pilots ran the same six papers in the real Docker sandbox and used the definitions of section 5.3 with a denominator of 6. Pilot 1 (`docs/PILOT_WEEK2.md`, 2026-09-30, GitHub's Linux runners) ran each paper's script directly from the repository root, on one Python version, with dependencies from its pip requirement files only. Pilot 2 (`docs/PILOT_2.md`, 2026-10-10, the owner's Windows machine with Docker Desktop and WSL2) ran each paper with its own run setup (section 5.2), on the Python version its repository names, with the same resource limits and the GPU off.

Install success: 6 / 6 (1.0) (Pilot 1: 1 / 6 (0.167)). Run success: 1 / 6 (0.167) (Pilot 1: 0 / 6 (0.0)). Pilot success: 1 / 6 (0.167) (Pilot 1: 0 / 6 (0.0)).

Measure	Pilot 1 (data/pilot_runs/)	Pilot 2 (data/pilot2_runs/)
Install success	1 / 6 (0.167)	6 / 6 (1.0)
Run success	0 / 6 (0.0)	1 / 6 (0.167)
Pilot success	0 / 6 (0.0)	1 / 6 (0.167)
Valid runs	6 / 6 (1.0)	6 / 6 (1.0)

Paper	Pilot 1	Pilot 2
paper_002	MISSING_DEPENDENCY (install)	MISSING_DEPENDENCY (run)
paper_003	MISSING_DEPENDENCY (install)	OTHER (run)
paper_004	MISSING_DEPENDENCY (run)	TIMEOUT (run)
paper_005	MISSING_DEPENDENCY (install)	OTHER (run)
paper_006	MISSING_DEPENDENCY (run)	success
paper_008	DATA_NOT_ACCESSIBLE (run)	DATA_NOT_ACCESSIBLE (run)

Failure category	Pilot 1	Pilot 2
MISSING_DEPENDENCY	5	1
DATA_NOT_ACCESSIBLE	1	1
HARDWARE_GPU_REQUIRED	0	0
TIMEOUT	0	1
SCRIPT_NOT_FOUND	0	0
LICENSE_BLOCKED	0	0
OTHER	0	2

What each Pilot 2 paper stopped at, decided by its first failure, with the log line that decided it:

Paper	Category	Phase	Deciding line
paper_002	MISSING_DEPENDENCY	run	ModuleNotFoundError: No module named 'lightgbm'
paper_003	OTHER (WRITE_TO_READ_ONLY_REPOSITORY)	run	OSError: [Errno 30] Read-only file system: '/workspace/repo/data/california_housing/build_X__quantile__mean__new__indices__0.pickle'
paper_004	TIMEOUT	run	status: TIMEOUT
paper_005	OTHER (OUT_OF_MEMORY)	run	status: OOM_KILLED
paper_006	success	n/a	n/a
paper_008	DATA_NOT_ACCESSIBLE	run	FileNotFoundError: [Errno 2] No such file or directory: './data/forest_cover_type/forest-cover-type.csv'

What changed between the pilots is the sandbox and the run setups, not the papers or the definitions. Pilot 1's dominant failure, dependencies that could not be installed, is gone: every paper now installs its dependencies, including the papers that are described only by a conda environment file and those that need a package built from source. What remains are obstacles that belong to the papers or to the resources given to them: an undeclared dependency (`paper_002`), a cache file that the paper writes into its data directory (`paper_003`, which

this pilot's frozen run setup mounts read-only), the time limit (`paper_004`, still training when it hit it), the memory limit (`paper_005`), and split files that were never published (`paper_008`). `paper_006` is the only paper that ran to completion. Per-paper times and peak memory are in `docs/PILOT_2.md` section 12.

The first Pilot 2 attempts were stopped mostly by defects in Repro Radar itself, which only running the tool on a second platform (Windows, against the Linux CI that existed then) and on real papers revealed: a CRLF container entrypoint, dependency markers evaluated for the host instead of the sandbox, a conda lock resolved instead of installed as pinned, a writable mount whose parent directory was owned by root, and others. Each was fixed in its own issue and PR, the paper was re-run, and every earlier attempt is kept with its reason (`data/pilot2_runs/first_attempts/README.md`, generated from the stored records; the list is in `docs/PILOT_2.md` section 12).

6.1a Pilot 3: the same papers with four settings changed

Pilot 3 (`docs/PILOT_3.md`, 2026-10-13) repeated Pilot 2 with four per-paper settings changed: a writable copy of the data for `paper_003`, 12 GiB of memory for `paper_005`, four hours for the run phase of `paper_004`, and `lightgbm` among the requirements of `paper_002`. **These settings were chosen after Pilot 2's failures were seen, so the change in the rates measures the effect of the settings, not an improvement of the papers.** Pilot 1 and Pilot 2 are unchanged.

Install success: 6 / 6 (1.0) (Pilot 1: 1 / 6 (0.167); Pilot 2: 6 / 6 (1.0)). Run success: 2 / 6 (0.333) (Pilot 1: 0 / 6 (0.0); Pilot 2: 1 / 6 (0.167)). Pilot success: 2 / 6 (0.333) (Pilot 1: 0 / 6 (0.0); Pilot 2: 1 / 6 (0.167)).

Measure	Pilot 1 (data/pilot_runs/)	Pilot 2 (data/pilot2_runs/)	Pilot 3 (data/pilot3_runs/)
Install success	1 / 6 (0.167)	6 / 6 (1.0)	6 / 6 (1.0)
Run success	0 / 6 (0.0)	1 / 6 (0.167)	2 / 6 (0.333)
Pilot success	0 / 6 (0.0)	1 / 6 (0.167)	2 / 6 (0.333)
Valid runs	6 / 6 (1.0)	6 / 6 (1.0)	6 / 6 (1.0)

Paper	Pilot 1	Pilot 2	Pilot 3
paper_002	MISSING_DEPENDENCY (install)	MISSING_DEPENDENCY (run)	DATA_NOT_ACCESSIBLE (run)
paper_003	MISSING_DEPENDENCY (install)	OTHER (run)	success
paper_004	MISSING_DEPENDENCY (run)	TIMEOUT (run)	OTHER (run)
paper_005	MISSING_DEPENDENCY (install)	OTHER (run)	OTHER (run)
paper_006	MISSING_DEPENDENCY (run)	success	success
paper_008	DATA_NOT_ACCESSIBLE (run)	DATA_NOT_ACCESSIBLE (run)	DATA_NOT_ACCESSIBLE (run)

Failure category	Pilot 1	Pilot 2	Pilot 3
MISSING_DEPENDENCY	5	1	0
DATA_NOT_ACCESSIBLE	1	1	2
HARDWARE_GPU_REQUIRED	0	0	0
TIMEOUT	0	1	0
SCRIPT_NOT_FOUND	0	0	0
LICENSE_BLOCKED	0	0	0
OTHER	0	2	2

What each Pilot 3 paper stopped at, with the log line that decided it:

Paper	Category	Phase	Deciding line
paper_002	DATA_NOT_ACCESSIBLE	run	ConnectionError: HTTPSConnectionPool(host='www.openml.org', port=443): Max retries exceeded with url: /api/v1/xml/data/list/limit/10000/offset/0/data_id/11,14,15,16,18,22,23,29,31,37,50,54,188,458,469,1049,1050,1063,1068,1510,1494,1480,1462
paper_003	success	n/a	n/a
paper_004	OTHER (OUT_OF_MEMORY)	run	status: OOM_KILLED
paper_005	OTHER (OUT_OF_MEMORY)	run	status: OOM_KILLED
paper_006	success	n/a	n/a
paper_008	DATA_NOT_ACCESSIBLE	run	FileNotFoundError: [Errno 2] No such file or directory: './data/forest_cover_type/forest-cover-type.csv'

One of the four changes removed its obstacle and led to a completed run (`paper_003`); the others exposed the next stop (`paper_002`: a request to the OpenML API with the network off; `paper_004`: the memory limit after the longer run phase; `paper_005`: the memory limit again, further into its grid search). The two papers whose settings did not change gave the same result as in Pilot 2. The first Pilot 3 attempt of `paper_002` was stopped by a defect in Repro Radar's own output limit and is kept with its reason (`data/pilot3_runs/first_attempts/README.md`).

Memory re-runs. The three papers that ended at a limit or completed with room to spare were run once more

with Pilot 3's settings and a memory limit of 50% of the host's RAM (`docs/MEMORY_RUNS.md`). These are not protocol runs, the memory limit differs, and no rate above was recomputed from them. By the stored records:

`paper_003`: COMPLETED at 6.0 GiB (peak 1.11 GiB) in Pilot 3, COMPLETED at 13.93 GiB (peak 1.09 GiB) in the memory re-run; `paper_004`: OOM_KILLED at 6.0 GiB (peak 6.0 GiB) in Pilot 3, COMPLETED at 13.93 GiB (peak 10.64 GiB) in the memory re-run; `paper_005`: OOM_KILLED at 12.0 GiB (peak 12.0 GiB) in Pilot 3, OOM_KILLED at 13.93 GiB (peak 13.93 GiB) in the memory re-run.

Read this way: with the larger limit `paper_004` ran to its end, so its Pilot 3 stop was the protocol's 6 GiB; `paper_003` did not need the extra memory; `paper_005` was still killed by the memory limit, later in its grid search. None of the re-runs is scored against a claim.

6.2 Can the numbers be checked? Claim extraction

Claim extraction	Value
Questions (6 papers)	18
Answers over both passes	36
Claims found	24 (all from a table: 24)
Quote guard: quote and value found in the paper text	17 / 24 (unverified: 7)
Passes A and B agree	18 / 18 (disagreements: 0)
Reviewed by the owner	accepted 13, rejected 0, pending 0
Gold set (pass A), scored questions	18
True positives / false positives / false negatives / true negatives	12 / 0 / 0 / 6
Precision / recall	1.0 / 1.0
Regression cases (must be true negatives)	paper_005:ep3602_auroc: TN, paper_005:ep3602_accuracy: TN, paper_008:higgs_auroc: TN

These figures describe the extraction pipeline, with three caveats. Pass B is independent in process but not in model: a fresh Claude Code session that saw only a blind packet found the same 12 values and the same 6 "not found" answers as pass A (`docs/AGENT_EXTRACTION.md` section 3), but both passes are the same model and the session was started by the agent that knew pass A. The candidate values in the gold set form were listed by the same agent that did the extraction, although the owner chose among them without seeing which one the agent had chosen, so a value the agent never found could enter the gold set only through the free "other" answer, which the owner did not use. And 18 questions over 6 papers is a small sample, so a perfect score is weak evidence. What the figures do show: every claim that was accepted has a quote that occurs in the paper's own text, a person decided on each one, and the two cases built to catch an invented value (a dataset and a metric that the paper does not contain) were answered "not found".

6.3 What the pilots do not show

No Pilot 2 run produced a metric that an accepted claim names. `paper_006` completed, but on the Bank dataset, while its accepted claims concern Arcene; the other five papers stopped before producing results. In Pilot 3 `paper_003` completed too and prints a test RMSE under the metric name `rmse`, which three accepted claims also use, but only for FT-Transformer; section 6.11 compares it with its claim. `cli.py score` can score a stored run against the accepted claims (`python cli.py score <paper> --from-run <run directory>`, #136), but until the claim map (#507, #509) the claims were matched by question ID and the extracted metrics by name, so for these runs its Tier 2 was not a measurement; sections 6.1 to 6.3 list no composite scores, and section 6.11 lists them before and with the map. The pilots measure whether the papers install and run, not whether their numbers reproduce.

- **Sources:** `docs/PILOT_WEEK2.md` section 11 and `docs/PILOT_2.md` section 12 (the generated blocks above are copied from `data/pilot_runs/` and `data/pilot2_runs/` by `scripts/report_numbers.py`); `data/pilot2_runs/first_attempts/` (every earlier attempt); `data/claims/` (guard report, A/B agreement, the owner's review decisions, the gold-set score); `docs/AGENT_EXTRACTION.md` (section 3, how independent the passes are).
- **Status:** written. Every number in this section is a generated block; `python scripts/report_numbers.py --check` and a test fail if one is edited by hand.

6.4 Checks made after the pilots

Small checks were made after the pilots. None of them changes a pilot rate; the first three are checks of the tool, and the seed runs and the GPU check of section 6.4a are described there.

A synthetic control paper (`data/synthetic_control/`, #313). A tiny repository with a deterministic classifier whose accuracy is known by construction. It was run twice in the real Docker sandbox and scored against a true and a false claim, so the scorer is tested on a case whose answer is known:

Metric `accuracy`, reproduced 0.9 in both runs; true claim 0.9, false claim 0.5. Two runs agree on status (true), exit code (true), and metrics (true); log files identical: 2 of 2.

Scored	True claim: Tier 2 / total	False claim: Tier 2 / total
Before the fix (a repository without requirements counted as a failed install)	0 / 36.75 (Needs Review)	0 / 36.75 (Needs Review)
After the fix	100 / 91.75 (Verified)	0 / 56.75 (Needs Review)

The control found a defect in Repro Radar's own scoring: a repository that lists no requirements has the install status SKIPPED, which the scoring step counted as a failed install, so the reproduced metric was ignored and both claims scored Tier 2 = 0. After the fix the true claim reaches Tier 2 = 100 and the false claim stays at 0 (the first result is kept in `result_before_fix.json`). The control tests the scorer; it is not a paper, has no training, data, or network, and says nothing about real papers. Tier 3 was not exercised by it (section 6.4a does that, on seed runs).

Repeated runs of the completed papers. The two papers whose runs completed have more than one stored run. Agreement is judged on the status, the exit code, and the metrics that the metric extractor found in the output:

Paper	Run	Status (exit)	Duration (s)	Peak memory (GiB)	Memory limit (GiB)	CPUs	Metrics
paper_003	Pilot 3 (data/pilot3_runs/paper_003)	COMPLETED (0)	1516.673	1.11	6.0	2.0	loss 0.106, rmse 0.473
paper_003	memory re-run (data/memory_runs/paper_003)	COMPLETED (0)	1497.443	1.09	13.93	2.0	loss 0.106, rmse 0.473
paper_003	memory re-run, second (data/memory_runs/paper_003_run2)	COMPLETED (0)	1390.574	1.03	13.93	2.0	loss 0.106, rmse 0.473
paper_003	CPU run (data/cpu_runs/paper_003)	COMPLETED (0)	465.777	1.15	13.93	8.0	loss 0.106, rmse 0.473
paper_006	Pilot 2 (data/pilot2_runs/paper_006)	COMPLETED (0)	3122.657	0.65	6.0	2.0	accuracy 0.8886, auroc 0.892
paper_006	Pilot 3 (data/pilot3_runs/paper_006)	COMPLETED (0)	3435.356	0.65	6.0	2.0	accuracy 0.8886, auroc 0.892
paper_006	memory re-run, second (data/memory_runs/paper_006_run2)	COMPLETED (0)	3268.898	0.65	13.93	2.0	accuracy 0.8886, auroc 0.892
paper_006	CPU run (data/cpu_runs/paper_006)	COMPLETED (0)	1119.834	0.64	13.93	8.0	accuracy 0.8886, auroc 0.892

`paper_003`: 4 runs, status identical, exit code identical, metrics identical (loss 0.106, rmse 0.473); 3 of 9 log files identical (line endings ignored) with the first run's; `paper_006`: 4 runs, status identical, exit code identical, metrics identical (accuracy 0.8886, auroc 0.892); 9 of 15 log files identical (line endings ignored) with the first run's.

The runs listed are not repetitions under one protocol: the runs of each paper were made at different commits of the tool (the `repro_radar_commit` of their records), the memory limit also differs for `paper_003` (`docs/MEMORY_RUNS.md`), and the CPU runs of `paper_003` and `paper_006` used 8 CPUs instead of 2 (`data/cpu_runs/`, `docs/CPU_RUNS.md`); the paper settings are the same. Within these limits the status, the exit code, and the metrics are the same in every run listed, while the log files differ where they contain clock times. The runs were not made with different seeds, so this is not the stability tier of the score (#48); the seed runs are in section 6.4a.

Memory re-runs. Section 6.1a and `docs/MEMORY_RUNS.md` show Pilot 3's settings with a larger memory limit: the limit alone decided the outcome of `paper_004`, and `paper_005` was killed again later in its grid search. They are not protocol runs.

A fresh run from a clean clone (#441). To check that the README is enough for a person who has only the public repository, a fresh clone was followed step by step to a completed run of `paper_006` and its score. The numbers below are read from the walk-through document, which is a hand-kept transcript of timed console steps; the run record itself is not in the repository (`sandbox_runs/` is not committed), and other work ran on the machine meanwhile, so the times are not those of an idle machine:

A fresh clone of the public repository followed through the README for `paper_006`: the run completed; total time from the clone to the end of `diagnose` 2,490 s, run phase 1,092.54 s; metrics `accuracy 0.8886, auroc 0.892`; accepted claims used: 2, none found in the output (the run reads another dataset); Tier 2 0, total 61.25, status Needs Review.

(This walkthrough was recorded before the claim map existed; with the map the same stored run gets Tier 2 70 and `Needs Review`, because both accepted claims have no counterpart in it and nothing was compared, section 6.11.)

This shows that the documented steps work end to end for one paper with the 8-CPU setting given on the command line; it does not show that the paper's numbers reproduce (its accepted claims belong to another dataset), and it is one paper on one machine.

Extended gold set. Eighteen more questions were added to the gold set (`docs/GOLD_SET_EXPANSION.md`):

Extended gold set: 18 new questions; pass A and the blind pass B agree on 18 of 18 (disagreements: 0); the owner's answers do not exist yet, so no precision or recall is computed or claimed.

The agreement of two passes by the same model, one of them blind, shows that the answers are reproducible by the model, not that they are right. The extraction accuracy of section 6.2 therefore still rests on the first 18 questions.

- **Sources:** `data/synthetic_control/`, `scripts/run_stability.py`, `docs/MEMORY_RUNS.md`, `data/gold_expansion/`.
- **Status:** written.

6.4a Seed runs for the stability tier, and a check with the GPU

Seed runs (#48, `docs/MULTI_SEED.md`). A paper is run once per seed only if it has a seed parameter of its own; nothing in a paper's code is changed. For `paper_003` the seed is the configuration file (`0.toml`, `1.toml`, `2.toml` differ only in their first line, `seed = N`); for `paper_006` it is the option `--set_seed` (1, 2, 3). Each seed is one real run in the sandbox, with the paper's Pilot 3 setting, 4 CPUs, and 50% of the host's memory; **these are not protocol runs** (the CPU and memory limits differ from the protocol's) and every record says so. The seeds were written into `data/multi_seed_settings.json` before any run. Two synthetic papers test the path on numbers that are known beforehand: one prints a metric that ignores its seed, the other draws its metric from `random.Random(seed)`; their mean and standard deviation were also computed on the host from the same seeds and compared with the summary of the sandbox runs.

Paper	Metric	Seed runs	Mean	Std (population)	Std (sample)	CV
synthetic_seed_free (synthetic)	accuracy	5	0.9	0	0	0.000%
synthetic_seeded (synthetic)	accuracy	5	0.7531	0.1079	0.1207	14.333%
paper_003	loss	3	0.1173	0.0087	0.0106	7.375%
paper_003	rmse	3	0.4693	0.0033	0.004	0.703%
paper_006	accuracy	3	0.892	0.0025	0.003	0.276%
paper_006	auroc	3	0.9027	0.0079	0.0097	0.879%

Paper	Successful seed runs	Tier 3 score	Basis	Decided by; score of each metric alone
synthetic_seed_free (synthetic)	5	100.0	measured: the metric with the largest CV decides	accuracy; accuracy 100.0
synthetic_seeded (synthetic)	5	40.0	measured: the metric with the largest CV decides	accuracy; accuracy 40.0
paper_003	3 of 3	65.0	measured: the metric with the largest CV decides	loss; loss 65.0, rmse 100.0
paper_006	3 of 3	100.0	measured: the metric with the largest CV decides	auroc; accuracy 100.0, auroc 100.0

The synthetic papers' summary equals the mean and standard deviation computed on the host from the same seeds: `synthetic_seed_free` yes, `synthetic_seeded` yes.

Read this way: on numbers that are known beforehand the path gives what it should (the Tier 3 column of the second table and the sentence under it): the synthetic paper that ignores its seed is in the top band, the one that draws from its seed is in the bottom band, and the sandbox summaries equal the host's calculation. For the two real papers, the first seed gave exactly the metrics of the earlier single runs of the paper (it uses the same configuration file or the option's default), and the metrics moved a little with the seed. `paper_006` stayed within the top band on both of its metrics. For `paper_003` the headline `rmse` varies little between seeds, but the extractor also finds a `loss`, the training loss printed by the code, which varies more; because the metric with the largest CV decides, `loss` sets the Tier 3 of `paper_003`, and `rmse` alone would give the top band (the last column). That is a consequence of the rule "the largest CV decides" and of what the extractor picks up from a log, not a finding about the paper. Because that was a weak basis for a score, the scorer was changed afterwards (#542): when an accepted claim is mapped to a run metric that has seed values, only that metric

decides Tier 3. The table below shows both rules; the "Tier 3 score" column of the generated table above is the summary file's own, older rule, which is kept as a record. The new rule is a default chosen by the assistant and the owner may revise it (`docs/PENDING.md`, D7). In both cases the measured value replaces the single-run baseline of 75.0, which was a placeholder.

Paper	Run metric of an accepted claim	Tier 3, largest-CV rule	Tier 3 by the scorer	Decided by
paper_003	rmse	65	100	rmse (the claimed metric)
paper_006	none (no accepted claim has a counterpart)	100	100	auroc (the largest CV)

What these runs do not show: each paper has three seeds, so a CV is a rough number; the runs are not protocol runs (4 CPUs and 50% of the host's memory, the records say so) and `paper_006` ran at the same time as `paper_003`, so their durations are not clean; only the paper's training seed varies (the data split of `paper_006` has its own option, left at its default) and the runs are CPU-only, so nothing is said about nondeterminism of a GPU or of a library; and Tier 2 was not exercised on the seed runs themselves (the one claim that the claim map pairs with a run metric, of `paper_003`, is compared on the stored single runs, section 6.11; the claims of `paper_006` concern another dataset, section 6.3).

A synthetic paper with the local GPU (#169). `scripts/gpu_paper_check.py` runs a small repository that pins `torch==2.8.0` (the default PyPI Linux wheel of that release is built for CUDA 12.8, which the RTX 5070 needs) through the same pipeline as a pilot paper: wheel bundling in the image, offline install, then the sandbox with `--gpus all`, no network, and a read-only root. The paper does a matrix product on the GPU and compares it with the CPU result.

Fact	Value
Requirement of the synthetic paper	torch==2.8.0
torch / CUDA build seen in the container	2.8.0+cu128 / 12.8
Device seen by torch	NVIDIA GeForce RTX 5070, capability (12, 0)
Tensor operation on	cuda:0; result equal to the CPU result: True
Run report gpu block	requested all, available True, used True; devices: NVIDIA GeForce RTX 5070 (driver 610.88, 12227 MiB); peak GPU memory 2530 MiB
Bundled wheels	3.65 GiB, bundling took 575.403 s
Sandbox time (install and run)	563.407 s
Checks passed	8 of 8

The tensor lived on the GPU, the result equalled the CPU result, and the isolation was unchanged. The paper's own output is the strong evidence; `gpu.used` in the run report is a heuristic (GPU memory in use rose during the run, and other programs on the host can raise it). This is one synthetic paper: it shows that the sandbox can give a modern PyTorch the GPU, not that any real paper benefits. The papers of the corpus pin PyTorch 1.x, which this GPU cannot use, and none was run with it.

- **Sources:** `data/multi_seed_runs/`, `data/synthetic_seeds/`, `data/gpu_check/`, `docs/MULTI_SEED.md`, `docs/ENVIRONMENT.md` (GPU section).
- **Status:** written.

6.5 Pilot 4: six papers the tool had not seen

Pilots 1 to 3 used the six papers that the project corrected before the pilots. Pilot 4 (`docs/PILOT_4.md`, written before any run) ran six **new** papers, chosen by rules that were fixed first and that never looked at whether a paper runs (`docs/CORPUS_EXPANSION.md`; 263 candidates were evaluated in a fixed hash order and six were selected). Each paper was run once with the command of its README as written, under fixed limits (2 CPUs, 50% of the host's RAM, a 14400 s run phase, no network), and a paper whose first attempt ended in an error could get one correction round, restricted to changes that the first attempt's logs justify. Tier 1 only: no claims were extracted and nothing was scored. The first attempt is the strict result, and both are shown. The table puts the four pilots side by side; **they are not the same conditions** (different papers, limits, and, above all, the earlier pilots ran hand-written run setups and Pilot 4 ran the README command alone).

Measure	Pilot 1	Pilot 2	Pilot 3	Pilot 4, first attempt	Pilot 4, after one correction round
Install success	1 / 6 (0.167)	6 / 6 (1.0)	6 / 6 (1.0)	2 / 6 (0.333)	3 / 6 (0.5)
Run success	0 / 6 (0.0)	1 / 6 (0.167)	2 / 6 (0.333)	2 / 6 (0.333)	2 / 6 (0.333)
Pilot success	0 / 6 (0.0)	1 / 6 (0.167)	2 / 6 (0.333)	1 / 6 (0.167)	1 / 6 (0.167)

Paper	README command	First attempt	After the correction round	Corrections
paper_101	python docs/check_styles.py	RAN_WITHOUT_INSTALLED_DEPENDENCIES (install): COMPLETED, exit 0	no correction round	none
paper_102	python code/examples/quick_start.py	success (COMPLETED, exit 0, 12.746 s)	no correction round	none
paper_103	python example.py	MISSING_DEPENDENCY (install): FAILED, exit 1	no correction round	none
paper_104	python src/prepare_data.py	DATA_NOT_ACCESSIBLE (run): FAILED, exit 1	no correction round	none
paper_105	python train.py --dataset Cora	MISSING_DEPENDENCY (install): FAILED, exit 1	no correction round	none
paper_106	python patch_train.py --batch_size 32 --aug True	MISSING_DEPENDENCY (run): FAILED, exit 1	MISSING_DEPENDENCY (run): FAILED, exit 1	add torch to the extra requirements (the repository lists no requirement file)

Failure category	First attempt	After the correction round
MISSING_DEPENDENCY	3	3
DATA_NOT_ACCESSIBLE	1	1
HARDWARE_GPU_REQUIRED	0	0
TIMEOUT	0	0
SCRIPT_NOT_FOUND	0	0
LICENSE_BLOCKED	0	0
OTHER	0	0

Read this way: with the README command alone, two of six new papers installed, two ran to exit code 0, and one (`paper_102`) did both, against six of six installs in Pilots 2 and 3, whose setups the project had prepared by hand. The five papers that did not succeed stopped at the kinds of obstacles seen before: requirements without a binary wheel (`torchnet` needs `visdom`; `torch-scatter` and its relatives; `tensorboardX` after `torch` was added), a download that the offline sandbox cannot make (`paper_104`), and one command that is a documentation checker and ran without installed dependencies (`paper_101`). The correction round could be used for one paper only (`paper_106`): for three the logs did not name a change that the protocol allows, and for two the first attempt did not end in an error. It raised the install rate from 2/6 to 3/6 and did not change the pilot success rate. The correction round also found a defect of the tool (extra requirements were ignored for a repository without a requirements file), which was fixed, and the earlier run is kept. With six papers this is an anecdote: it says that a README command alone is a weak recipe for these papers in this sandbox, not how often it is for machine learning papers in general; the pool was biased and drifted from tabular data (`docs/CORPUS_EXPANSION.md` sections 5 and 7).

6.5a External reference articles: ReScience C

To have an outside reference for the score (articles that reviewers found reproducible), the project fixed rules before looking at the pool and applied them to the articles of ReScience C (`docs/EXTERNAL_VALIDATION.md`). The rules were relaxed once after the first result was seen (rule E4 accepts `python -m`, `make`, and notebook commands, section 11 of that document, marked as written after the result), and the protocol for running the selected articles was written (`docs/EXTERNAL_VALIDATION_PROTOCOL.md`).

Articles of ReScience C evaluated: 65; selected: 0. Rule that stopped each article (the first failed rule): E4 25, E2 23, E3 11, E1 5, E6 1.

No article passed, so **no external-validation run was made** and nothing in this report depends on one. The most common stop is the rule that the README must state a command the pipeline can run; the pool's articles mostly run through other forms (shell scripts, a `python3` line, a Makefile that is absent). Adding such forms would be a further change to the rules after the result was seen, and was not made. What to do about the reference is an open question for the owner and the mentor.

6.6 Field analysis over twelve papers (exploratory)

`docs/FIELD_ANALYSIS.md` fixes, before the numbers were computed, what counts as human intervention and how the autonomy rate and three descriptive factors are calculated; `scripts/field_analysis.py` computes them from the stored records. The pool is the six seed papers and the six Pilot 4 papers. The first protocol run is

Pilot 1 (seed papers) or the Pilot 4 first attempt; the final one is Pilot 3 or Pilot 4 after the correction round.

Pool	Run	Setup success	Run success	Pilot success	Concluded autonomously	Succeeded autonomously
seed papers	first	1 / 6	0 / 6	0 / 6	0 / 6	0 / 6
seed papers	final	6 / 6	2 / 6	2 / 6	0 / 6	0 / 6
expansion papers	first	2 / 6	2 / 6	1 / 6	6 / 6	1 / 6
expansion papers	final	3 / 6	2 / 6	1 / 6	5 / 6	1 / 6
all 12 papers	first	3 / 12	2 / 12	1 / 12	6 / 12	1 / 12
all 12 papers	final	9 / 12	4 / 12	3 / 12	5 / 12	1 / 12

Papers with at least one claim accepted by a person (not part of the autonomy rate): seed papers 6 / 6, expansion papers 0 / 6, all 12 papers 6 / 12.

Read this way: the setup, run, and pilot success of the seed papers rose between their first and final run because of settings that people chose after the failures were seen (section 6.1a), and the autonomy rate of the seed papers is 0 by construction, since their command and setup were chosen by a person. Among the expansion papers, the autonomy rate counts a paper as concluded autonomously when no person chose its command or setup, no correction round was made, and no change to the tool was made for it (the definition, `docs/FIELD_ANALYSIS.md` section 5); it counts only human actions that left a file, so it is an upper bound. The factor table (documentation checklist, code age, declared dependencies) is in `docs/FIELD_ANALYSIS.md` section 8.5. It is a table, not a test: with twelve papers and crude measures no relationship between a factor and success is claimed, and the two sub-pools were chosen differently and are not comparable.

6.7 Diagnosis layer: explaining a run, and the difference between two

The diagnosis layer (`docs/DIAGNOSIS.md`, `src/diagnosis/`) is rule-based. For one stored run it classifies the failure with the categories of Pilot 1 and writes a report in which every sentence names the file and key it was read from, an unknown value is written as unknown, and no cause is named beyond the quoted log line (`python cli.py diagnose <run directory>`; `docs/DIAGNOSIS_EXAMPLES.md` holds the reports of every stored run of Pilots 1 to 4, the memory runs, and the CPU runs). For two runs of one paper it lists the recorded differences (CPUs, memory limit, seed, command, image, tool commit, package versions) and keeps "same metrics, different time" apart from "different metrics" (`docs/DIAGNOSIS_COMPARISONS.md`). An optional language-model step is off by default and cannot change a number; it was tested with a mock client only.

The classifier was checked against labels written by hand from the error lines of the stored runs (and against invented probes):

Set	Items	Category correct	Name of an OTHER correct	Phase correct	All levels correct	Wrong
Real runs	46	46 / 46	10 / 10	36 / 36	46 / 46	none
Synthetic probes	12	12 / 12	5 / 5	12 / 12	12 / 12	none

The labels are not blind (the labeler had read the classifier's rules), the real runs are the logs the rules came from, and the set is dominated by missing packages, so the first row is an upper bound for new papers. The probes were written to find gaps and found them: the rules have no name for a permission error, a GPU out of memory, or a full disk, they treat a missing `.pt` file as a plain missing file and not as data, and they do not count an HTTP error or a refused connection as a network failure (`docs/DIAGNOSIS_VALIDATION.md` gives the reason for each). The table above is from after #424 closed those six gaps with one rule each, without editing the labels (before: real runs 45 / 46, probes 7 / 12 at all levels); because the rules were written from these very lines, the table now only shows that rules and labels agree. No real language model was used, so nothing is claimed about the quality of a model-written restatement.

6.8 Candidate papers run once in the sandbox

The owner collected candidate papers with a public repository (`docs/CANDIDATES.md`). The ones that the lists call ready, fixable or two-step were run once each by a protocol that was merged before the first run (`docs/CANDIDATE_RUNS.md`): the command of the list, the sandbox, no network, no retry, no repair after a result. Nothing of the candidates is in the seed corpus.

Selected candidates: 14; run once in the sandbox: 13; passed: 0 (none). Pilot ratio 0 / 14, Wilson 95 % interval 0.0 % to 21.5 %. Cause of the failures (rules of the failure classifier): MISSING_DEPENDENCY (the installation failed) 6, MISSING_DEPENDENCY 3, WRITE_TO_READ_ONLY_REPOSITORY 2, COMMAND_INCOMPLETE 1, DATA_NOT_ACCESSIBLE 1, MISSING_FILE 1. This is a selected list, not a sample; it is not a success rate.

The list was chosen by other people for being easy to run, so it is a selected list and the share that passed is not

a success rate of anything; a repair (another Python version, an extra requirement, a writable copy, a data file) was forbidden by the protocol, so the result does not say that these papers cannot be reproduced. In round 1 no candidate passed, so none was used for a claim extraction or a demo.

The owner then allowed a second, separate round in which a candidate may be repaired within five narrow rules (an extra or pinned requirement, a pin the README states, outputs sent to the writable folder by an argument the code has, the correct folder, an open data download the README names), at most twice, never by editing the paper's code or shortening its run (`docs/CANDIDATE_RUNS_ROUND2.md`, merged before the first round-2 run). The first attempt (round 1) stays as it is:

First attempt without repair (round 1): 0 of 14, Wilson 95 % interval 0.0 % to 21.5 %. With repair (round 2, the same 14): 1 of 14, 1.3 % to 31.5 % (cand_002). Considered in round 2 (those and the risky candidates that qualified): 1 of 15, 1.2 % to 29.8 %; run in the sandbox at least once: 6. This is a selected list, not a sample; it is not a success rate.

Most of the candidates stopped because their failure needs something the rules forbid (a hard-coded write into the read-only repository, a pinned package that has no wheel). The one that passed ran the repository's own example and printed numbers that occur in no table of its paper, so it is a pass of the pattern and not a reproduction; its blind claim extraction found no value to propose (`docs/CANDIDATES.md`, Round 2 results). Nothing is accepted.

6.9 A tolerance probe on real runs

To see where the Tier 2 decision changes, the stored completed runs of `paper_003`, `paper_006` and `paper_102` were scored with claims made by hand from the run's own value, moved up and down by a known share (`docs/TOLERANCE_PROBE.md`, `scripts/tolerance_probe.py`):

Tolerance 0.05. Hand-made claims moved from the value of a real run; first deviation at which the status of the true claim changes:

Paper	Runs	Metrics	Downward	Upward
paper_003	4	2	-20 % (Verified -> Partially Verified)	+20 % (Verified -> Partially Verified)
paper_006	4	2	-10 % (Verified -> Partially Verified)	+10 % (Verified -> Partially Verified)
paper_102	1	12	-10 % (Verified -> Partially Verified)	+10 % (Verified -> Partially Verified)

Accepted claims of the owner that match a probed run by question ID: 0 of 13.

These are hand-made claims, not an error of a real paper. The probe shows the tolerance as implemented (a relative band measured from the claimed value, plus an absolute allowance for a claim between 0 and 1 that makes small values lenient); it does not measure how often the scorer is wrong on real papers, and the rates of false positives and false negatives in the real world have not been measured. The absolute allowance of 0.05 for a claimed value between 0 and 1 is kept as it is (a default chosen by the assistant, the owner may revise, `docs/PENDING.md`); it is a known limit, because it lets a small value such as 0.0167 match a reproduced 0.06. No claim that the owner accepted matches a probed run by its question ID.

6.10 An automatic search that checks a run's numbers against the paper's text

A third, larger round (`docs/AUTO_SEARCH_ROUND3.md`, protocol merged before the first search) took repositories from GitHub's search by topic, applied the static rules of the demo set in the order of a hash, ran each candidate that passed once in the sandbox exactly as its README says (run test K1), and then looked up the numbers that the run printed in the text of the paper (match test K2: at least two different numbers, one of them on a page with a table caption). Candidates that failed were not repaired or replaced. Topic stages were added to the protocol, each before its search and without looking at a candidate, when the earlier stages were used up (`scripts/auto_search3.py`):

Candidates checked against the static rules: 3399; run once in the sandbox: 135. Passed K1 (the run test): 2 of 135, Wilson 95 % interval 0.4 % to 5.2 %. Passed K2 (numbers of the output found in the paper's text): 0 of 135, Wilson 95 % interval 0.0 % to 2.8 %. Passed both: 0 of 135, Wilson 95 % interval 0.0 % to 2.8 % (none). Cause of the K1 failures: run 101, install 27, no_metric_line 5. The search stopped because: pool_used_up. This is a selected, non-representative list; it is not a success rate, and a K2 pass would be a match candidate, not a verified result.

K2 is a string match and can match by chance; a pass would be a match candidate and not a verified result, and it would need a person to read the page and the line that were stored. The list is selected and not representative:

the shares describe the candidates that were run, under these rules, and are not the reproducibility of a field. No claim is accepted by this round.

6.10a A fourth round with a match test that reads scales (K2 v2)

The first positive control of the match test failed for a reason that was known: `paper_006`'s run prints the AUROC as a fraction (`0.933`) and the paper prints a percent (`93.3`). Before a fourth search the protocol was extended (`docs/AUTO_SEARCH_ROUND4.md`, written after the result of the third round was seen and merged before the first run): K2 v1 stays as it was, and a second test K2 v2 is added that also looks up a fraction as a percent and the reverse, rounds to three significant digits, and reads decimal commas and thousands separators; a converted match counts only on a page with a table caption. With the stored runs, `paper_006` passes K2 v2 (one converted match plus a coincidental `256`) and the random-data example `paper_102` does not. The search used topic stages 8 to 21 under the same static rules and run test as round 3 (`scripts/auto_search4.py`):

Candidates checked against the static rules: 2648; run once in the sandbox: 107. Passed K1 (the run test): 3 of 107, Wilson 95 % interval 1.0 % to 7.9 % (a4_019, a4_045, a4_105). Passed K2 v1 (numbers of the output found in the paper's text, same spelling): 1 of 107, Wilson 95 % interval 0.2 % to 5.1 %. Passed K2 v2 (the same with fraction/percent conversion and rounding): 1 of 107, Wilson 95 % interval 0.2 % to 5.1 % (a4_067). Passed K1 and K2 v1: 0 of 107, Wilson 95 % interval 0.0 % to 3.5 %. Passed K1 and K2 v2: 0 of 107, Wilson 95 % interval 0.0 % to 3.5 %; passed K1 and either K2: 0 (none). Cause of the K1 failures: run 66, install 31, no_metric_line 4, harness 2, staging 1. The search stopped because: time_limit_9h. This is a selected, non-representative list; it is not a success rate, and K2 v2 can match by chance: a pass is a match candidate, not a verified result.

K2 v2 passed the same single candidate as K2 v1, a fuzzing script that failed the run test and whose matched numbers are integers and progress counters (my reading of the stored lines, not checked by a person). The conversion of scales did not find a match that v1 missed in this round, and the two run-test passers of round 3, re-evaluated with K2 v2 from their stored output, still have no number in common with their papers. The list is selected and not representative; a K2 pass is a match candidate, not a verified result; K2 v2 can match by chance. No claim is accepted by this round.

6.11 What the scorer decides on real runs, and the pilot as an estimate

`docs/REAL_VERDICTS.md` (`scripts/real_verdicts.py`, #494) puts the completed stored runs of the real papers next to the claims a person accepted and scores them with `src/score_run.py`, without writing a database. The table is its summary; the document holds one row per run and claim, with the reason for each decision. Since #507 and #509 the command line pairs each accepted claim with the run's metric through `data/claim_metric_map.json` (`docs/CLAIM_METRIC_MAP.md`), by model, dataset, measure and split and never by the closeness of the values: 1 of the 13 accepted claims is mapped, 12 have no counterpart in the stored runs. The document keeps the result before the mapping next to the result with it. The map is a proposal that the owner may revise. The pairing of the script (`aligned`) and its evidence are written in the script and agree with the map on the one mapped claim.

Paper	Stored runs scored	What the scorer says	Meaning of the decision
paper_003	6	before the claim map (claims by question ID): Tier 2 0.0 in 6 of 6; with the map (same model, dataset and metric): claimed 0.459, reproduced 0.473, relative difference 3.05 %, Tier 2 100, Verified in 6 of 6 runs	comparable for one of three claims (the run trains FT-Transformer only); one claim and one number, not a confirmed reproduction
paper_006	3	before the claim map: Tier 2 0.0 in 3 of 3; with the map: 6 of 6 claim rows have no counterpart (Arcene numbers, the run reads another dataset), Tier 2 70 (neutral baseline), Needs Review	none comparable; nothing was compared, so the label is Needs Review
paper_004	1	before the claim map: Tier 2 0.0; with the map: no counterpart, Tier 2 70 (neutral baseline), Needs Review	none comparable: the stored metrics are hyperparameters (lr, weight_decay)
paper_102	1	no accepted claim; Tier 2 70 (neutral baseline), Needs Review	no comparison was made
synthetic control (known answer)	2	true claim: Tier 2 100, Verified in 2 of 2 runs; false claim: Tier 2 0, Needs Review in 2 of 2 runs	the only known-answer rows

Before the claim map the claims never matched a run's metrics: Tier 2 was 0.0 in 10 of the 10 real runs that have accepted claims (question IDs against extracted metric names). With the map a claim is compared in 6 of the 10

runs (the one claim of paper_003); in the others no accepted claim has a counterpart in the run and Tier 2 is the neutral baseline. K2 (numbers of the output found in the paper's text) passed 6 of 6 runs of paper_003, 0 of 3 runs of paper_006, 0 of 1 runs of paper_102 and 0 of 1 runs of paper_004. For paper_003 the claimed value 0.459 is printed in the paper (page 7), while the run's final rmse 0.473 is not printed in it.

Read this way. First, before the claim map the decision path of the command line never compared a real claim: claims were keyed by question ID and the extracted metrics by name, so Tier 2 was zero and the label Needs Review for every real run that has accepted claims. Second, with the map the one meaningful comparison is paper_003: the same model, dataset and metric, a small difference (3.05 %) inside the 5 % tolerance, so Tier 2 is 100 and the label Verified; whether that small a difference should count as reproduced is a tolerance question (section 6.9), and this single claim and single number is not a confirmed reproduction. The Six stored runs of it give the same last RMSE line. Third, the other real runs cannot be compared at all (another dataset, hyperparameters stored as metrics, no accepted claim): with the map their claims have no counterpart and leave Tier 2, which then is the neutral baseline of 70. Before #540 paper_004 and paper_006 therefore got the label Partially Verified from the build tiers alone, as paper_102 did, a label without a comparison; since #540 a run with nothing compared is Needs Review (the table above), and its total still contains the baseline. Fourth, K2, the check that a run's printed numbers occur in the paper, was expected to pass on paper_003, paper_006 and paper_102 as a positive control. It passed on paper_003 only, and there by coincidence: the numbers it matched are training-log values that the paper's tables print for other models, while the run's final value is not printed in the paper. The run of paper_006 prints AUROC as a fraction where its paper prints percent, and the quick start of paper_102 works on generated toy data, so its output is in no table. A positive control for K2 therefore still needs a real run whose output prints a table value of its paper. The synthetic control is the only row whose answer is known, and there the scorer is right.

The pilot as an estimate. The second claim, how many papers run, is an estimate. Pilot success with Wilson 95 % intervals, from the stored records of the field analysis (section 6.6):

Pool	Run	Pilot success	Wilson 95 % interval
seed papers	first	0 / 6	0.0 % to 39.0 %
seed papers	final	2 / 6	9.7 % to 70.0 %
expansion papers	first	1 / 6	3.0 % to 56.4 %
expansion papers	final	1 / 6	3.0 % to 56.4 %
all 12 papers	first	1 / 12	1.5 % to 35.4 %
all 12 papers	final	3 / 12	8.9 % to 53.2 %

The intervals are wide because the samples are small (six seed papers, six new papers), the setups of the seed papers were prepared by hand, and the new papers were chosen by rules from a biased pool. The figures are not a rate of a field.

- **Sources:** docs/REAL_VERDICTS.md, data/real_verdicts/verdicts.json (scripts/real_verdicts.py), data/field_analysis/result.json, data/claims/review_decisions.json, docs/BUG_HUNT.md section 3 (the same inputs checked for defects).
- **Status:** written.

6.12 Why the runs failed

docs/FAILURE_STATISTICS.md (scripts/failure_statistics.py, #546) counts the first failure of every stored run by simple text rules on the log line that decided it: the 24 attempts of Pilots 1 to 4, the candidate list (round 1 and the repair round) and the runs of the automatic searches of rounds 3 and 4. Each group has its own column and the groups are not added together.

Category of the first failure	Pilots 1 to 4 (24)	Candidate list, round 1 (14)	Candidate list, repair round (15)	Automatic search, round 3 (135)	Automatic search, round 4 (105)
passed	4	0	1	2	3
missing_python_module	4	3	1	61	40
install_package_not_found	5	6	4	26	24
read_only_write	1	2	3	13	8
data_or_network_missing	5	2	3	10	8
ran_no_metric_line	0	0	1	5	4
timeout	1	0	0	0	0
out_of_memory	3	0	0	0	0
api_key_missing	0	0	0	3	1
wrong_argument	0	0	0	6	1
not_run_command_incomplete	0	1	1	0	0

Category of the first failure	Pilots 1 to 4 (24)	Candidate list, round 1 (14)	Candidate list, repair round (15)	Automatic search, round 3 (135)	Automatic search, round 4 (105)
not_run_judged_not_repairable	0	0	1	0	0
staging_failed	0	0	0	0	1
ran_without_installed_dependencies	1	0	0	0	0
other	0	0	0	9	15

In the two automatic searches and in round 1 of the candidate list, a missing Python module and an installation that found no matching package or wheel are the two most frequent first failures; in the repair round and the pilots the picture is flatter (installation, missing data or network, and in the pilots also out-of-memory kills), and those groups are small. These are counts of why selected runs failed, not a success rate. The lists are selected: the automatic searches rejected every repository whose README or requirement files mention `gpu` or `cuda` (rule S3), so a GPU requirement cannot appear in them, and Pilot 3 used settings chosen after the failures of Pilot 2 had been seen. A category is the kind of the first deciding line, not a proven root cause, and `other` collects lines that no rule matched.

- **Sources:** `data/failure_statistics.json`, `docs/FAILURE_STATISTICS.md` (`python scripts/failure_statistics.py --check`).
- **Status:** written.

6.13 The dashboard in its final form

The local dashboard (`src/web/`, `docs/DECISIONS.md` D-006: localhost only, no new library, the earlier layout kept) was extended so that a label is never shown without its reason (#548). Beside each label the list says whether a claim was compared and, when none was, that Tier 2 is a baseline of nothing compared; the page of a paper has a "Why this label" section with the claims that were compared, the claims without a counterpart and the reason for each, the tolerance rule, the difference in percent, where Tier 3 came from, the stored run, and, for a failed run, the failure category with the log line it was read from; the synthetic control is listed apart as "not a paper: known-answer control"; a page `/failures` shows the stored failure statistics with the selection-bias warning; and the footer states how the database was built, that the scores come from `src/scoring.py`, that the numbers are not a success rate, and what `Needs Review` means. The pages show stored values only: the reasons are stored beside each score when it is computed (`scripts/build_demo_db.py`, `cli.py score`). Screenshots from a fresh clone are in `docs/evidence/demo3/`. The dashboard does not add evidence: it shows what the scorer decided on stored runs.

- **Sources:** `src/web/`, `src/score_explanation.py`, `docs/evidence/demo3/`, `docs/DEMO_SCRIPT.md` (Step 1).
- **Status:** written.

7. Limitations and threats to validity

- **Small sample.** Six papers, one counted run each (the seed runs of section 6.4a are extra, not counted, runs of two papers): the results are anecdotal, not statistical. The corpus is tabular deep learning and sequence kernels, and it was corrected and reduced before the pilots (section 5.1); no paper was replaced because it failed (#166).
- **The run setups were written by us.** The entry command, working directory, data, build, extra requirements, and version constraints were chosen by Repro Radar. A paper that runs shows that it *can* be run this way, not that its README alone is enough. The per-paper settings were frozen for Pilot 2, so an obstacle caused by a setting (the read-only data mount of `paper_003`) stays in the result and was applied in Pilot 3 (section 6.1a).
- **Defects were fixed during the pilot.** The attempts ran at several Repro Radar commits. The counted runs are at two commits that differ only in the measurement tools (`docs/PILOT_2.md` section 12), and every earlier attempt is kept.
- **Substitutions and unpinned versions.** The CPU build of `faiss` replaces conda's CUDA build (`paper_004`), and a package that a requirement file leaves unpinned was installed at the newest version that fits, not the version the authors used.
- **Resources.** 2 CPUs, 6 GiB, 3600 seconds per phase, and no network were chosen by us; a paper that needs more fails here although it might run on the authors' hardware (`paper_005` at the memory limit, `paper_004` at the time limit). The memory re-runs (section 6.1a) show that the limit alone decided one of these: `pape`

`r_004` completed with 14265m. They are single runs on one machine, and the smallest sufficient limit was not searched for. The counted runs are CPU-only; the machine's RTX 5070 needs a PyTorch built for CUDA 12.8 or newer, which the PyTorch 1.x pins of several papers cannot use.

- **One platform per pilot.** Pilot 1 ran on Linux runners and Pilot 2 on one Windows machine; some failures may depend on the platform, which is also how the Windows-only defects of the tool were found.
- **No numerical reproduction was confirmed.** Before the claim map no run matched an accepted claim through the command line (claims were keyed by question ID, the extracted metrics by name); with the map only one real run, `paper_003`, prints a number for a claim, close to it but not equal (section 6.11). Tier 2 (agreement with the paper's numbers) was therefore exercised on real results only in that one comparison, on the synthetic control paper (section 6.4), which tests the scorer and not a paper, and on hand-made claims (section 6.9).
- **The README command mostly does not produce the paper's table.** In the automatic search no run's output matched its paper's text, and none of the listed candidates, at the first attempt or with repair, printed a table value of its paper (sections 6.8 and 6.10 give the counts). A run that completes is therefore usually not an attempt to reproduce a table, and this limits what any pilot rate says: it is a finding about README commands in this sandbox, not a measure of how reproducible the papers are.
- **A total without a comparison.** A paper with no accepted claim, or whose accepted claims all have no counterpart in its run (`paper_102`, `paper_004`, `paper_006`), gets a neutral Tier 2 of 70. Since #540 its label is `Needs Review`, never `Verified` or `Partially Verified`, but its total score still contains the neutral baseline (for example 86.5 for `paper_006`), so the number is higher than the evidence behind it. Only one real claim, of `paper_003`, is compared at all.
- **No positive control for K2.** The match test of the searches has no real run that is known to print a table value of its paper; the only pass among the real runs, `paper_003`, is by coincidence of dense numbers (section 6.11), so no K2 pass can be read as a reproduction.
- **The absolute allowance of 0.05.** A claimed value between 0 and 1 matches when it is within 0.05 in absolute terms, which makes small values lenient (section 6.9). It was kept (default K-B, a choice of the assistant that the owner may revise) and is a known limit.
- **Selection bias.** Every list is selected: the six seed papers and the six Pilot 4 papers by hand or by rules, the candidate list by other people for being easy to run, and the automatic searches through rules that removed repositories mentioning a GPU. The failure counts of section 6.12 and all rates are about these lists, not about research code in general.
- **Stability rests on three seeds of two papers.** The seed runs of section 6.4a are not protocol runs (the CPU and memory limits differ from the protocol's), use three seeds each, and cover only the two papers that complete; a CV from three runs is a rough number, and when no accepted claim maps to a run metric the metric that decides Tier 3 can still be a side number such as a training loss (since #542 the claimed metric decides when there is one; a default of the assistant, which the owner may revise). The synthetic papers show that the path measures a known difference, not that a real paper is stable. Before them the repeated runs of section 6.4 (different memory limits or tool commits, no different seeds) showed only that the metrics repeated.
- **Field analysis.** Twelve papers from two differently chosen sub-pools, definitions written after the pilot outcomes were known, and an autonomy rate that counts only human actions that left a file (section 6.6): exploratory, no test, no relationship claimed.
- **Diagnosis layer.** Rule-based and tested on stored runs of this project; the hand labels are not blind, the real runs are the logs the rules came from, and the probes found gaps that were closed afterwards with rules written from those same lines (section 6.7).
- **No language-model path was run with a real model.** The optional language-model step of the diagnosis layer was tested with a mock client only, and the API extraction code (#156) was never run with a key (decision D-004); claim extraction was done by an agent under fixed rules (section 4.3).
- **GPU.** One synthetic paper only; no real paper was run with the GPU, and the seed papers pin PyTorch 1.x, which the GPU cannot use.
- **External reference.** The ReScience C selection evaluated 65 articles and selected none (section 6.5a), so the score has no outside reference yet.
- **Pilot 4 is six new papers, one attempt each.** The sample came from a biased pool and drifted from tabular data, the README command is the first `python` line and is not always the paper's experiment (a

documentation checker, a data preparation step), and the correction round was limited to what the logs justify, so a person with the code might have corrected more. The Pilot 4 rates are not comparable to the earlier pilots' rates, whose setups were prepared by hand (section 6.5).

- **No second gold set.** The 18 added questions have two agreeing passes but no answers from the owner, so the accuracy of the extraction rests on the first 18 questions only and the second set could not be used to measure it (section 6.4).
- **Extraction.** It is agent-assisted; the two passes are the same model, although pass B was made blind; the gold set was built from candidates listed by the same agent; and the quote guard proves that a quote occurs in the page text, not that the number means what the question asks (`docs/AGENT_EXTRACTION.md`). `paper_005`'s only accepted claim (EP300 average kernel speedup, 16) is a follow-up question that is not among the 18 gold-set questions, so it is not part of the precision and recall figures.
- **Conclusion bounds.** From these results one can say that the sandbox and the run setups made the papers installable and exposed what blocks each of them; one cannot say how reproducible tabular machine learning is in general.
- **Sources:** `docs/PILOT_2.md` section 12 (its Limits, and the proposals); `docs/PILOT_WEEK2.md` section 9; `docs/AGENT_EXTRACTION.md`; `docs/CORPUS_VERIFICATION.md` sections 5 and 6; `data/pilot2_runs/first_attempts/`.
- **Status:** written.

8. Lessons learned

- **Run the tool on a second platform early.** The real-Docker acceptance checks passed in CI throughout (while CI existed); two defects only appeared on a Windows host (a CRLF entrypoint, dependency markers evaluated for the host), and several more only when real papers were run.
- **A conda environment file is a lock.** The first answer to the conda failures was a resolver pass over the pins (#227/#228); it failed on a source-only dependency and on conda's `mkl` packages. Installing the pins as they are, without resolving, was the correct model (#229).
- **Fix the tool, not the experiment.** Freezing the per-paper settings and allowing only fixes to Repro Radar's own code (`docs/PILOT_2.md` section 9) is what let a failure be told apart: the tool's defects were fixed and re-run, while the paper's own cache write and memory use stayed in the result.
- **Keep every attempt and say why it was replaced.** The earlier attempts show which obstacles were the tool's and which the papers'; the index is generated from the stored records, so the story is not retold from memory.
- **Generate the numbers and test the generation.** The generated blocks and their tests caught a stored file that carried a local path and a log line that a color code hid from the categorizer.
- **Label with the definitions, then look at the rules.** The hand-labeled check of the classifier (section 6.7) was labeled by someone who had read the rules, so it is not blind; the probes written to find gaps found them. Report both, and keep the numbers of the check that cannot lose.
- **A noisy side number can decide a score.** The first real seed run showed that the metric with the largest CV was a training loss; the summary shows which metric decided and what each metric alone would give, and since #542 the claimed metric decides when the claim map names one (section 6.4a).
- **Look at what the scorer does with real data before trusting its tests.** The tests of `cli.py score` pass because they store the metric under the question ID; a real run stores it under the name the extractor found, and before the claim map the two never met (section 6.11). The stored runs were scored only when a document asked for them.
- **What we would change next:** the claims and the run's metrics now meet through a claim map (#507, #509; a proposal the owner may revise); choose each paper's command so that it produces the metrics its accepted claims name (`docs/PILOT_2.md` section 12), and find one real run whose output prints a table value of its paper, as a positive control for the match test (section 6.11). The four per-paper changes that `docs/PILOT_2.md` proposed (a writable data directory for `paper_003`, more memory for `paper_005`, more time for `paper_004`, `lightgbm` for `paper_002`) were applied in Pilot 3 (section 6.1a).
- **Sources:** `docs/PILOT_2.md` section 12; `data/pilot2_runs/first_attempts/README.md`; `docs/DECISIONS.md` (D-008 and the earlier decisions); the issues named above.
- **Status:** written.

9. Conclusion and future work

What the evidence supports. The generated rates in sections 6.1 and 6.1a show install success, run success, and pilot success of the three pilots side by side. A fourth pilot on six new papers, run from their README commands alone, installed fewer papers than Pilots 2 and 3 did (section 6.5), which indicates that part of the earlier installs came from the setups written by the project; with six papers it is an indication, not a measurement. Part of that movement is the work on Repro Radar's own sandbox and run setups, and part of it is four per-paper settings that were chosen after the failures were seen (section 6.1a), so it is not an improvement of the papers. The failures that remain have named causes in the stored records: a dataset that is not accessible (`paper_008`), a request to the OpenML API with the network off (`paper_002`), and the memory limit (`paper_004`, `paper_005`). The claim extraction pipeline produced claims that each have a verbatim quote in the paper and an owner decision (section 6.2). The stability tier was run on real seed runs for the first time (section 6.4a) and the diagnosis layer explains stored runs and the difference between two (section 6.7). The dashboard of the project (`src/web/`) shows stored scores, badges, and audit reports from the database only; a fresh database (`python cli.py init-db`) holds no score, so the dashboard shows "Not scored" for every paper until `cli.py score` is run. `python scripts/build_demo_db.py` builds a separate database from the stored real runs, in which every paper has the score of its best stored run (`docs/evidence/demo3/`, the final form of section 6.13); those scores come from the scorer through the claim map: Tier 2 comes from a comparison for `paper_003` only and is the neutral baseline for the papers whose accepted claims have no counterpart (their label is `Needs Review`), and they are not validations (section 6.11).

What the two claims rest on. The decision on a known example (the synthetic control and the hand-made claims) is supported; the decision on a real paper is not, because no real paper has a known answer in this work and the one real comparison is close but unconfirmed (section 6.11). The count of papers that run is an estimate with wide intervals, not a rate.

What it does not support. Section 7 applies: six papers, one run each, setups written by the project, and no confirmed numerical reproduction. One cannot say from this work how reproducible tabular machine learning is, or that the rubric's agreement tier and stability tier work on real results.

Future work, in the order the evidence suggests.

1. The question IDs of the claims and the extracted metric names now meet through the claim map (#507, #509); choose each paper's command so that it prints the metrics its accepted claims name, and score such a run with `cli.py score`; this is what would exercise Tier 2 on more than one real claim, and a run whose output prints a table value of its paper would also serve as the missing positive control of the match test (section 6.11).
2. Decide how much memory `paper_004` and `paper_005` need (the pilot only shows that the values chosen in advance were not enough) and whether `paper_002` may fetch its OpenML data.
3. Repeat the blind second pass with another model, or with the owner starting it, so that the two passes are also independent in model.
4. Run more seeds per paper, and on more papers than the two that complete, so that the stability tier (#48) rests on more than three runs; decide whether the claimed-metric rule of #542 is the right one (section 6.4a).
5. Try the optional language-model step of the diagnosis layer with a real model, and label new, unseen failure logs to measure the classifier again (section 6.7: its six known gaps are closed, so the stored set no longer measures it), before anything is said about either.
6. Decide what outside reference to use for the score now that the ReScience C selection found none (section 6.5a).

- **Sources:** sections 6 and 7 of this report and the records they cite; `docs/PROGRESS.md` for the dashboard (D1 to D4, `docs/evidence/d1` to `d4`); issues #31 to #36, #48, and #413 and the files named in sections 6.4a to 6.7.
- **Status:** written.

10. References and reproducibility statement

References

Each reference was looked up as described in `docs/INTRODUCTION_SOURCES.md` (section 1) and `docs/report/`

`RELATED_WORK_SOURCES.md` (section 1). Only bibliographic data and abstracts were read for most of them, not full texts; the tables name what was read, and what was not verified.

1. J. F. Claerbout and M. Karrenbach (1992). *Electronic documents give reproducible research a new meaning.* SEG Expanded Abstracts, 601-604. doi:10.1190/1.1822162
2. J. B. Buckheit and D. L. Donoho (1995). *WaveLab and reproducible research.* Lecture Notes in Statistics, 55-81. doi:10.1007/978-1-4612-2544-7_5
3. D. L. Donoho (2010). *An invitation to reproducible computational research.* Biostatistics 11(3), 385-388. doi:10.1093/biostatistics/kxq028
4. V. Stodden, F. Leisch, and R. D. Peng (Eds.) (2014). *Implementing Reproducible Research.* CRC Press. doi:10.1201/b16868
5. M. Baker (2016). *1,500 scientists lift the lid on reproducibility.* Nature 533(7604), 452-454. doi:10.1038/533452a
6. J. Pineau et al. (2021). *Improving Reproducibility in Machine Learning Research (A Report from the NeurIPS 2019 Reproducibility Program).* JMLR 22(164), 1-20. (The JMLR volume and pages were checked at jmlr.org; the arXiv preprint 2003.12206 matched on title and first author.)
7. P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger (2018). *Deep Reinforcement Learning That Matters.* AAAI 32(1). doi:10.1609/aaai.v32i1.11694; arXiv:1709.06560
8. O. E. Gundersen and S. Kjensmo (2018). *State of the Art: Reproducibility in Artificial Intelligence.* AAAI 32(1). doi:10.1609/aaai.v32i1.11503
9. E. Raff (2019). *A Step Toward Quantifying Independently Reproducible Machine Learning Research.* arXiv:1909.06674 (the preprint is cited; the conference version was not looked up).
10. C. Collberg and T. A. Proebsting (2016). *Repeatability in computer systems research.* Communications of the ACM 59(3), 62-69. doi:10.1145/2812803
11. J. F. Pimentel, L. Murta, V. Braganholo, and J. Freire (2019). *A Large-Scale Study About Quality and Reproducibility of Jupyter Notebooks.* MSR 2019, 507-517. doi:10.1109/MSR.2019.00077 (title and bibliographic data only; Crossref returned no abstract).
12. A. Trisovic, M. K. Lau, T. Pasquier, and M. Crosas (2022). *A large-scale study on research code quality and execution.* Scientific Data 9(1), 60. doi:10.1038/s41597-022-01143-6
13. C. Boettiger (2015). *An introduction to Docker for reproducible research.* ACM SIGOPS Operating Systems Review 49(1), 71-79. doi:10.1145/2723872.2723882; arXiv:1410.0846
14. X. Bouthillier et al. (2021). *Accounting for Variance in Machine Learning Benchmarks.* arXiv:2103.03098 (the preprint is cited; the conference version was not looked up).

Reproducibility statement

- **Code and data.** The repository holds the code, the six-paper corpus (`data/seed_papers.json`), every stored run record (`data/pilot_runs/`, `data/pilot2_runs/`, `data/pilot3_runs/`, `data/pilot4_runs/`, `data/multi_seed_runs/`, `data/synthetic_seeds/`, `data/gpu_check/`), and the claim files (`data/claims/`). **No release is tagged yet** (the release issue, #45, is open), so the version of this report is the commit of the pull request that merges it.
- **Re-run the checks that need no Docker.** `pip install -r requirements-dev.txt`, then `pytest tests/` and `ruff check .`; `python scripts/report_numbers.py --check` fails if a number of this report differs from its stored source; `python scripts/verify_sources.py` repeats the lookups of the related-work sources (needs the network).
- **Re-run the pilots.** They need Docker. The images are built with `docker build -f docker/Dockerfile.sandbox -t repro-radar-sandbox:latest .` (`docs/ENVIRONMENT.md`; the Python 3.8 and 3.9 images use `--build-arg PYTHON_VERSION=...`), and each paper runs with `python scripts/run_pilot.py <paper_id>` (Pilot 2) or with `--pilot3` (Pilot 3, `docs/PILOT_3.md`). Pilot 3 ran on one Windows machine with Docker Desktop and WSL2 at the commit recorded in `data/pilot3_runs/environment.txt`. A re-run on other hardware may differ, in particular for the memory limits. The seed runs and the GPU check ran on the same machine (`--memory 50% --cpus 4`), and their records say that they are not protocol runs.
- **Re-run the later checks.** `python scripts/synthetic_seeds.py` (two synthetic papers, five seeds each, needs Docker), `python scripts/run_multi_seed.py paper_003 --pilot3 --memory 50% --cpus 4`

(one real run per seed; `--summarize-only` rebuilds the summary from the stored folders), `python scripts/gpu_paper_check.py` (needs Docker and an NVIDIA GPU, about 3.7 GiB of wheels), `python cli.py diagnose <run directory>`, and, without Docker, `python scripts/field_analysis.py --check`, `python scripts/evaluate_diagnosis_classifier.py --check`, `python scripts/diagnosis_examples.py --check`, and `python scripts/diagnosis_comparisons.py --check`.

- **See the dashboard.** `python cli.py init-db`, then `python -m src.web` serves `http://127.0.0.1:8000` on the local machine only. With the real stored runs: `python scripts/build_demo_db.py`, then `python -m src.web --db-url sqlite:///data/dashboard_demo.db`. The decisions on the stored runs: `python scripts/real_verdicts.py --check` (no Docker; K2 needs the papers' PDFs in `data/pdfs/`, which are not committed, so only `--render` and `--check` run from the repository alone). The failure counts: `python scripts/failure_statistics.py --check`; the page `/failures` of the dashboard shows them.
- **What is not reproducible from the repository alone.** The datasets that the papers download (they are fetched before a run, `scripts/fetch_data.py`), the live responses of Crossref and arXiv, and any result that depends on the speed or memory of the machine.

AI assistance statement

This work was compiled with AI assistance. The software, the experiments and the text of this report were prepared with an AI assistant (Claude, Anthropic). The AI assistant did not decide which claims enter the scoring: only claims that a person accepted are scored. Every number in the results comes from a stored record through a script (`scripts/report_numbers.py`), and what could not be verified is listed in the limitations (section 7).

- **Sources:** `docs/INTRODUCTION_SOURCES.md`, `docs/report/RELATED_WORK_SOURCES.md`, `docs/report/related_work_verification.json`, `README.md`.
- **Status:** written.